

Humans versus AI: A Multi-Dimensional Characterization of Code Quality and Security

TODO Author  

TODO Affiliation, Country

Abstract

Background. TODO: Context and motivation.

Aims. TODO: Research goals and questions.

Method. TODO: Dataset scale, tooling, statistical methods, and analysis pipeline.

Results. TODO: Main empirical findings with quantitative evidence.

Conclusions. TODO: Implications and broader interpretation.

2012 ACM Subject Classification Software and its engineering → Software defect analysis; Software and its engineering → Software testing and debugging

Keywords and phrases TODO: comma-separated keywords

Digital Object Identifier 10.4230/LIPIcs.TODO.2025.TODO

Funding *TODO Author:* TODO funding

Acknowledgements TODO: Acknowledgments text. Include funding sources, data providers, or reviewers as appropriate.

1 Introduction

AI-assisted programming tools have become a standard part of modern software development. Systems such as GitHub Copilot, Devin, and Cursor now automate tasks ranging from code completion to pull request creation, operating with minimal human oversight. The scale of this adoption is substantial: the AIDev dataset documents over 980,000 AI-generated contributions spanning more than 116,000 repositories [3]. We draw on this dataset to examine how AI authorship affects code quality across a broad range of open-source projects.

While automation accelerates development, it also introduces quality concerns that differ in character from those typically associated with human-authored code. Existing research shows that AI-generated code exhibits higher error rates than human-authored code [1, 4, 2], yet several practical questions remain unanswered: (1) which violation categories and severity levels predominate in AI-generated code, (2) how structural complexity metrics relate to defect density and whether these relationships differ between AI and human-authored code, and (3) which programming languages face the largest quality gaps and whether static type systems reduce AI-associated risks.

We address these questions through a large-scale empirical investigation of 19,539 files drawn from the AIDev dataset, applying three complementary analyses: language-specific linters to assess functional correctness, structural complexity measurements to assess architectural quality, and security scanners to characterize CWE patterns. All analyses are stratified by programming language, enabling direct comparison of AI-generated and human-authored code across multiple quality dimensions. The main contributions of this work are as follows:

- 1. Large-scale empirical assessment of AI-generated versus human-authored code quality.** We analyze 19,539 files across six programming languages using a multi-dimensional framework covering functional, structural, and security quality dimensions, enabling systematic characterization of quality differences across language families and authorship types.



© TODO Author Names;
licensed under Creative Commons License CC-BY 4.0

TODO Conference Name (TODO 2025).

Editors: TODO; Article No. TODO; pp. TODO:1–TODO:4



Leibniz International Proceedings in Informatics

LIPICs Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

2. **Evidence that type safety reduces AI-associated quality gaps.** Statically-typed languages (Java, C#, TypeScript) exhibit significantly narrower violation density gaps between AI-generated and human-authored code (1.32x to 1.66x) compared to the 2.34x gap observed in dynamically-typed languages. Chi-square analysis confirms this pattern ($p < 0.001$), suggesting that compiler-enforced type checking intercepts a class of errors that otherwise propagate in dynamic languages.
3. **Identification of divergent structural complexity patterns in AI-generated code.** We provide empirical evidence that AI-generated code exhibits an ‘inverted’ quality profile characterized by high cyclomatic complexity alongside low structural coupling, a pattern statistically distinct from human-authored code ($p < 0.05$). This finding indicates that the structural composition of AI-generated code does not conform to assumptions underlying conventional quality metrics.

2 Related Work

2.1 TODO: Subsection Title

TODO: Synthesize prior work thematically. Compare findings across studies and identify unresolved gaps. Separate pre-AI era findings from AI era findings where relevant.

2.2 TODO: Subsection Title

TODO: Position the current work relative to the closest prior studies. Highlight what this paper addresses that prior work does not.

3 Experiment Design

3.1 Research Questions

This study investigates the following research questions:

RQ1: TODO: Research question 1.

RQ2: TODO: Research question 2.

RQ3: TODO: Research question 3.

3.2 Dataset

TODO: Describe the dataset. Include repository selection criteria, scale (number of repositories, files, lines of code), language coverage, and any filtering or exclusion decisions.

3.3 Tooling

TODO: Describe the tools used for static analysis, metric extraction, or vulnerability detection. Justify tool choices and note known limitations.

4 Methodology

4.1 Repository Selection

TODO: Describe repository selection criteria. Explain filtering strategy, exclusion rules, and language mapping decisions.

4.2 Metric Extraction

TODO: Explain the metrics collected (e.g., cyclomatic complexity, lines of code, defect density). Justify why file-level or function-level analysis is appropriate.

4.3 Statistical Analysis

TODO: Explain the statistical tests used and why they were chosen (e.g., nonparametric tests due to non-normal distributions). Describe effect size measures. Clarify the unit of analysis and multiple comparison corrections if applicable.

5 Results

5.1 RQ1: TODO Question Summary

TODO: Begin with a direct answer to RQ1. Present evidence before interpretation. Reference tables and figures naturally. Discuss distributions, variability, and language-specific behavior where relevant.

5.2 RQ2: TODO Question Summary

TODO: Begin with a direct answer to RQ2.

5.3 RQ3: TODO Question Summary

TODO: Begin with a direct answer to RQ3.

6 Discussion

6.1 Summary of Findings

TODO: Synthesize findings across all research questions. Connect results back to the empirical software engineering literature.

6.2 Implications for Practitioners

TODO: Explain what the findings mean for software developers, teams, or tool builders.

6.3 Implications for Researchers

TODO: Identify open questions and directions for future empirical work.

6.4 Threats to Validity

6.4.0.1 Construct Validity.

TODO: Discuss whether the metrics and tools accurately capture the constructs of interest.

6.4.0.2 Internal Validity.

TODO: Discuss confounds and alternative explanations for the observed differences.

108 6.4.0.3 External Validity.

109 TODO: Discuss the generalizability of findings across languages, domains, and repository
110 types. Address dataset representativeness and language imbalance.

111 6.4.0.4 Tooling Limitations.

112 TODO: Discuss false positives, false negatives, and known limitations of the static analysis
113 or metric extraction tools used.

114 — References —

- 115 1 CodeRabbit. State of AI vs human code generation report 2025. Technical report, CodeRabbit,
116 December 2025.
- 117 2 Raphael Khoury, Anderson R. Avila, Jacob Brunelle, and Bhaskar Mitra Camara. On the secur-
118 ity vulnerabilities of text-to-SQL models. In *Proceedings of the 2023 IEEE/ACM International*
119 *Conference on Software Engineering*, 2023. Study cited in: [https://www.scworld.com/news/ai-](https://www.scworld.com/news/ai-will-create-a-tidal-wave-of-buggy-vulnerable-software)
120 [will-create-a-tidal-wave-of-buggy-vulnerable-software](https://www.scworld.com/news/ai-will-create-a-tidal-wave-of-buggy-vulnerable-software).
- 121 3 Hao Li, Haoxiang Zhang, and Ahmed E. Hassan. The rise of AI teammates in software
122 engineering (SE) 3.0: How autonomous coding agents are reshaping software engineering, 2025.
123 AIDev-pop dataset Version 3 (August 2025 snapshot). URL: [https://arxiv.org/abs/2507.](https://arxiv.org/abs/2507.15003)
124 [15003](https://arxiv.org/abs/2507.15003), arXiv:2507.15003.
- 125 4 Yujia Rao, Sridhar Chimalakonda, and Chakkrit Tantithamthavorn. Security weaknesses of
126 Copilot generated code in GitHub. *arXiv preprint arXiv:2310.02059*, October 2023. URL:
127 <https://arxiv.org/abs/2310.02059>, arXiv:2310.02059.